

Assignment 4

Francesco Gnan, mail: gnanf@usi.ch

Deadline: 17 January 2024 - 10.00am

Conversational model with Transformers

1 Data (40 pts)

1. In lines 399-411 we download the two datasets and we inspect them using the function `read_and_print` which prints out the first `n` lines. We get the following output:

```
Movie Conversations:
u0 +++$+++ u2 +++$+++ m0 +++$+++ ['L194', 'L195', 'L196', 'L197']
u0 +++$+++ u2 +++$+++ m0 +++$+++ ['L198', 'L199']
u0 +++$+++ u2 +++$+++ m0 +++$+++ ['L200', 'L201', 'L202', 'L203']
u0 +++$+++ u2 +++$+++ m0 +++$+++ ['L204', 'L205', 'L206']
u0 +++$+++ u2 +++$+++ m0 +++$+++ ['L207', 'L208']

Movie Lines:
L1045 +++$+++ u0 +++$+++ m0 +++$+++ BIANCA +++$+++ They do not!
L1044 +++$+++ u2 +++$+++ m0 +++$+++ CAMERON +++$+++ They do to!
L985 +++$+++ u0 +++$+++ m0 +++$+++ BIANCA +++$+++ I hope so.
L984 +++$+++ u2 +++$+++ m0 +++$+++ CAMERON +++$+++ She okay?
L925 +++$+++ u0 +++$+++ m0 +++$+++ BIANCA +++$+++ Let's go.
```

In the `movie_conversations.txt` file, each line appears to represent a conversation between two users (u0 and u2) in a movie (m0). The lines are represented as lists of dialogue identifiers (['L194', 'L195', 'L196', 'L197']). In the `movie_lines.txt` file each line contains a unique identifier, the user involved (u0 or u2), the movie (m0), the character speaking (BIANCA, CAMERON), and the actual dialogue.

2. In lines 415-457 we use function `get_first_two_sentences` to create a list of pairs, each of them containing pairs of sentences (list of string pairs). For each conversation we choose to add just the very first two sentences. We print out the first two elements of this list:

```
[['They do not!', 'They do to!'], ['I hope so.', 'She okay?']]
```

3. In lines 460-512 we define two different functions to tokenize the sentences at a word level: `tokenization` and `tokenization.2`. In the following we choose to use the second one that uses the provided function `clear_punctuation` to remove special characters. So,

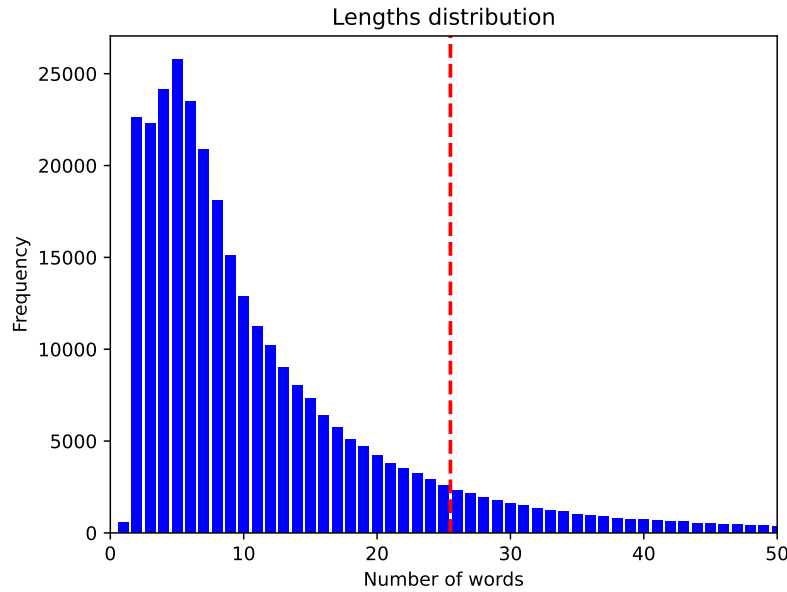


Figure 1: Distribution of the lengths of the sentences.

for each pair of sentences it creates a list of words. The special token '`<SOS>`' identifies the beginning of the answer. We report as an example the two first elements of the generated list of tokenized sentences:

```
[['they', 'do', 'not', '!', '<EOS>', '<SOS>', 'they', 'do', 'to', '!',  
  '<EOS>'],  
 ['i', 'hope', 'so', '.', '<EOS>', '<SOS>', 'she', 'okay', '?', '<EOS>']]
```

4. In lines 516-578 we remove all the pairs where there exists a sentence that is longer than a certain length `max_length`. We decide this threshold by looking at the distribution of the lengths of the sentences. Function `length_distribution` creates a dictionary where the lengths are the keys and their frequencies are the corresponding values. We show this distribution in Figure 1. The dashed red line indicates the threshold we choose: 25 words maximum. Afterwards, with function `remove_pairs` we remove all those pairs that contain at least a sentence longer than the fixed threshold. We check that before this operation our dataset contained 152356 pairs and after that 123214.
5. We save this result in pickle format in lines 580-581.
6. In lines 587-645 we remove those pairs of sentences that contain rare words. In detail, with function `words_frequency` we create an histogram of words frequencies and we exclude those that appear less times than a fixed minimum number. In Figure 2 we show for each word its corresponding frequency to make evident that in our dataset there are strings that appear very frequently and others that don't.

Function `delete_words` is then used to delete from our dataset all the pairs that contains a word whose frequency is less than the chosen threshold 50. We print out the difference

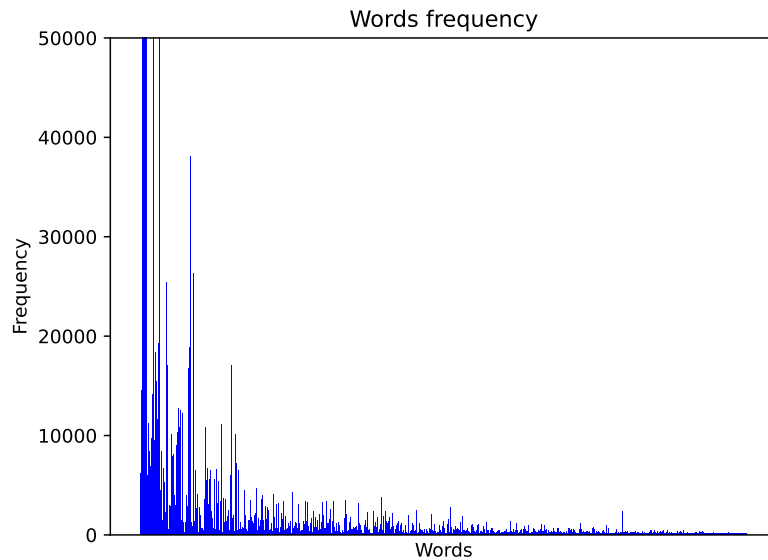


Figure 2: Word frequencies.

in size of the dataset before and after this operation:

```
Pairs count before eliminating words: 123214
Pairs count after eliminating words: 31407
```

7. Once again in lines 648-649 we save the list `filtered_pairs` in pickle format.
8. In lines 655-665 we randomly sample a subset of 20000 pairs and we split it into a `train_data` (80%) and `val_data` (20%).
9. We complete the class `Vocabulary` (lines 27-76). In particular, we fill the gaps in functions `add_word` and `add_sentence`. The latter simply get a sentence as input and calls the former to add each word in the given sentence to the two vocabularies `word2index` and `index2word`. In lines 669-679 we use this class to create a `train_vocab` and a `val_vocab`.
10. We complete the class `Dataset` (lines 91-129). As far as the `__init__` function is concerned, we use the two inputs to initialize the attributes of the class. The `__len__` function returns the length of the list of pairs of sentences we use as dataset. Finally, `__getitem__` creates two lists of indices both for the question and the answer; it uses the '`<EOS>`' tokens to distinguish them. It returns two tensors (type `torch.tensor`) `question` and `answer` that contain word indices. In lines 683-689 we define a `collate_fn` function that automatically pads the sequences in each batch. We recycled from the previous Assignment. Afterwards, in lines 692-699 we define two `Dataloader` objects, `train_loader` and `val_loader` respectively, using the class `Dataset` and the function `collate_fn`. We choose a batch size equal to 32.

2 Model & Tools for training (35 pts)

1. In Transformer models Positional encoding is used to learn information about token positions in the input sequence. Concerning the provided function, the constructor takes the dimension of the model (`d_model`), dropout rate (`dropout`), and the maximum length of the sequence (`max_len`) as parameters. It creates a positional encoding matrix (`pe`) using sine and cosine functions.

The forward method adds the positional encoding to the input tensor `x`. It first checks if the length of the sequence is less than the specified `max_len` to avoid indexing errors; then, it adds the positional encoding to the input tensor and applies dropout.

2. Considering the class `TransformerModel`, we complete the `__init__` method as shown below:

```
def __init__(self, vocab_size, d_model=512, pad_id=0, encoder_layers=6,
             decoder_layers=6, dim_feedforward=2048, num_heads=8, dropout_p=0.1):
    super().__init__()
    self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=
                                  pad_id)
    self.pos_encoder = PositionalEncoding(d_model, dropout_p)
    self.transformer = nn.Transformer(
        d_model=d_model,
        nhead=num_heads,
        num_encoder_layers=encoder_layers,
        num_decoder_layers=decoder_layers,
        dim_feedforward=dim_feedforward,
        dropout=dropout_p,
        batch_first=True
    )
    self.linear = nn.Linear(d_model, vocab_size)
    self.vocab_size = vocab_size
    self.pad_id = pad_id
    self.num_heads = num_heads
```

When we define the transformer layer, the `batch_first` parameter is a boolean argument that determines the ordering of the input tensors. Specifically, it defines whether the first dimension of the input tensors represents the batch size or the sequence length. If `batch_first` is set to `True`, the input tensors are expected to have the shape (`batch_size`, `sequence_length`, `features`). This means that the batch size is the first dimension of the input tensor. If `batch_first` is set to `False` (the default), the input tensors are expected to have the shape (`sequence_length`, `batch_size`, `features`), where the sequence length is the first dimension. Moreover, we choose (`d_model`, `vocab_size`) as dimensions for the linear layer since the output should be a probability distribution over the vocabulary.

3. Concerning the `create_padding_mask` function, we call the python operation `padding_mask = (x == pad_id)`. It is an element-wise equality check between the input sequence `x` and the specified `pad_id`. The result is a boolean tensor where each element is `True` if the corresponding element in `x` is equal to `texttt{pad_id}`, and `False` otherwise.
4. The forward is already provided. However, we changed line 241 (now commented) with line 240 because `dtype=torch.bool` produces some errors during training. `src_pad_mask`

and `tgt_pad_mask` are created using `create_padding_mask` to mask out padding tokens in the source and target sequences. Then, source and target sequences are embedded using the model's embedding layer and positional encoding is applied to capture positional informations. `memory_key_padding_mask` is set equal to `src_pad_mask`, ensuring that attention is not given to padded elements in the source sequence during the self-attention mechanism. Finally, the transformer layer is applied to the source and target sequences with the specified masks and its output is passed to the linear layer so as to generate the model's prediction.

5. Using 0/-inf masks aligns better with the softmax operation. It ensures that the attention mechanism appropriately assigns low probabilities to masked positions. In fact, for positions that are supposed to be ignored (masked), setting the corresponding values to -inf ensures that their resulting softmax probabilities approach zero.

3 Training (35 pts)

1. In lines 263-321 we define the `train` function. Within each epoch we put source and target to device (GPU or CPU), in the forward pass we exclude '`<EOS>`' from the decoder input (line) and '`<SOS>`' from target. To avoid an eventual exploding gradient issue, we apply gradient clipping. For each epoch we collect the train and validation average loss value. Finally, we apply a scheduler at the end of each epoch to dynamically adjust the learning rate during the training process and improve convergence and generalization. In Figure 3 we show the training and validation loss averages through the epochs. We observe an evident overfitting, so the model is not expected to generalize well on unseen data. To reduce this loss behaviour we tried to add L2 regularization (`weight_decay` parameter of the optimizer) and we applied early stopping. This is why we trained the model for just 10 epochs.
2. In lines 324-381 we define the new function `train_ga` that applies gradient accumulation technique. It is equivalent to the `train` function, but we added the following lines just after the backward pass:

```
accumulation_counter += 1
if accumulation_counter % accumulation_steps == 0:
    optimizer.step()
    accumulation_counter = 0
```

In other words, we update weights after accumulating gradients for a specified number of steps, allowing us to use smaller batches and consume less GPU memory. In Figure 4 we show the results of this training. Once again we observe overfitting, but at least this time the two losses are smoother.

4 Evaluation (10 pts)

- In lines 799-823 we implement greedy strategy (`generate_answer_greedy` function), to generate some answers to our questions: at each step, we pick the word that is the more likely to be the next one. In particular, we use `np.argmax` to get the words whose probability (softmax) is highest.

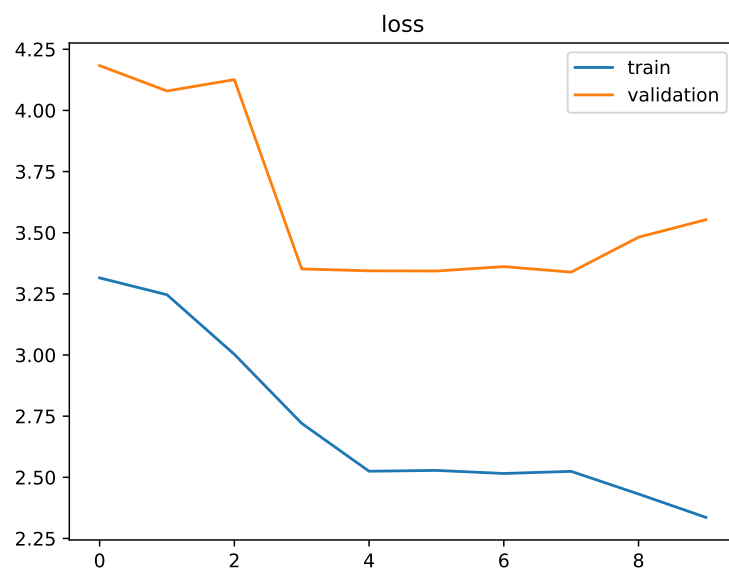


Figure 3: Standard training.

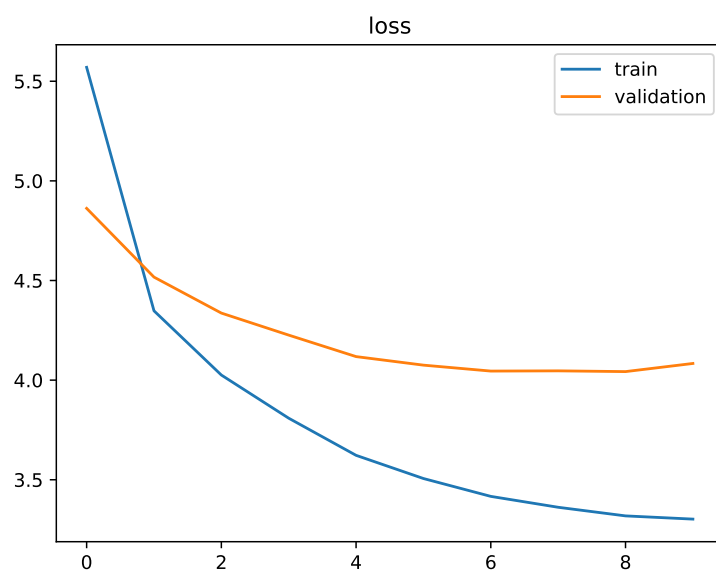


Figure 4: Training with gradient accumulation.

- In lines 825-850 instead we implement top-k sampling strategy. We let the model generate a probability distribution over the vocabulary (using softmax) and we pick just the top k values and sample the next word among them.
- We finally apply these two functions to generate answers to some given questions. We report the result below:

```

Question 1: ['how', 'old', 'are', 'you', '?']
Greedy Answer: ['<SOS>', 'i', "don't", 'know', '.', '<EOS>']
Topk Answer: ['<SOS>', 'no', 'the', 'little', 'think', 'the', 'hell',
              'to', 'have', '?']
-----

Question 2: ['what', 'is', 'your', 'name', '?']
Greedy Answer: ['<SOS>', 'i', "don't", 'know', '.', '<EOS>']
Topk Answer: ['<SOS>', "i'm", 'think', 'you', "don't", 'think', '<EOS>']
-----

Question 3: ['do', 'you', 'know', '?']
Greedy Answer: ['<SOS>', 'i', "don't", 'know', '.', '<EOS>']
Topk Answer: ['<SOS>', 'i', 'know', '?', "i'm", 'think', 'you', 'think',
              ', '<EOS>']

```

We observe that the greedy strategy produces always the same answer. This means that its implementation is wrong but we could not fix this problem.

Bonus

- We define a function `train_ga_hf` that uses the HuggingFace function `Accelerate`. We implement it in lines 884-964. We do not report the training results, since the loss function we get is very high (≈ 10). We conclude that probably our implementation is not correct.